



Repaso Lenguaje C y Memoria dinámica

REPASO LENGUAJE C	2
TIPO PUNTERO	2
FUNCIONES	2
CADENAS	3
ARREGLOS	3
REGISTROS	4
MEMORIA DINAMICA	4
ESTRUCTURAS DINÁMICAS	6

REPASO LENGUAJE C

TIPO PUNTERO

Recordemos que una variable de tipo puntero almacena una dirección de memoria.

Cuando se declara un puntero se reserva espacio para albergar una dirección de memoria, pero **no para almacenar el dato al que apunta el puntero**.

El espacio de memoria reservado para almacenar una variable de tipo puntero es siempre el mismo (4 bytes) independientemente del tipo de dato al que se apunte.

Los operadores ***** y **&** se utilizan para operar con direcciones de memoria. Son unarios (afectan sólo a un operando).

- ✓ el operador **&** permite obtener la dirección de una variable, y
- ✓ el operador ***** es el complementario de éste, pues al evaluarse devuelve el contenido de una dirección de memoria, por lo tanto el operando tiene que ser un puntero. El operador ***** sirve también para declarar una variable de tipo puntero a un tipo de dato determinado.

🔗 Ejemplo

```
int m=1, n=2, z;  
int *p, *q;                // p y q son punteros a enteros  
  
p = &m;                    // p recibe la dirección de m  
*p = 4;                    // sería incorrecto p=4; equivale m=4;  
printf("%d %p", *p, p);    // contenido y la dirección de m  
  
q = &n;                    // q recibe la dirección de n  
z = *q;                    // equivale z = n;  
printf("%d %d", *p, *q);  
  
p = q;  
printf("%d %d", *p, *q);
```

FUNCIONES

- ✓ Todo programa en C está formado por una o mas funciones (al menos *main()*).
- ✓ No se pueden definir funciones anidadas.
- ✓ Desde una función se puede invocar a otras, inclusive a ella misma (recursividad).
- ✓ **El mecanismo de paso de parámetros es por valor.**

La declaración de la cabecera debe anteceder a la propia definición de la función.

🔗 Sintaxis: *tipo_dato* nombreFuncion (*lista_tipo_argumentos*);

dónde *tipo_dato* es el tipo de dato devuelto por la función (en caso de obviarse el mismo se toma el tipo **int** por defecto). Para evitar malas interpretaciones es conveniente explicitarlo.

Cuando la función no devuelve ningún valor ("procedimiento"), se indica por medio de la palabra reservada **void** (sin valor).

lista_tipo_argumentos representa los tipos de datos de los argumentos que pueden ir acompañados o no de los nombres de los argumentos. Una función que no necesita argumentos se informa mediante la palabra **void** puesta entre los paréntesis.

🔗 Ejemplo

```
int maximo(int a, int b);  
void saludo();          /* o void saludo (void); */
```

Los parámetros de una función pueden ser valores (**pasaje por valor**) o direcciones (**pasaje por referencia**)

En el pasaje de parámetros por **Valor** las modificaciones en los valores de los parámetros formales de la función no afectan a las variables/constantes/expresiones del medio que hizo la llamada.

Si se quiere conseguir el efecto lateral de afectar a las variables del medio de llamada, hay que inducir el paso por **Referencia**, pasando direcciones de variables (&) en vez de sus valores. Entonces, se usan los parámetros formales desreferenciados (*) dentro de la función.

PASAJE POR VALOR	PASAJE POR REFERENCIA (DIRECCION)
<pre>void doble(int i) { i *= 2; }</pre>	<pre>void doble(int *i){ *i *=2; }</pre>
<pre>void main() { int n=5; printf("%d\n", n); // 5 doble (n) ; printf("%d\n", n); // 5 }</pre>	<pre>void main() { int n=5; printf("%d\n", n); // 5 doble (&n) ; printf("%d\n", n); //10 }</pre>

Agrupamiento de los prototipos

Es recomendable agrupar todos los prototipos de las funciones de un programa en uno o varios archivos de cabecera (.h) que luego se incluirán como encabezado en los programas que hagan referencia a las funciones allí declaradas.

CADENAS

👉 **string.h** : algunas funciones son

- ✓ **strlen(cad)** : devuelve la longitud de la cadena *cad* sin el \0.
- ✓ **strcpy(cad1, cad2)** : copia el contenido de *cad2* en *cad1*. Sería incorrecto *cad1 = cad2*. Devuelve un *char **. Variante: **strncpy(cad1, cad2, n)**.
- ✓ **strcat(cad1, cad2)** : concatena el contenido de *cad2* en *cad1* (sin controlar que haya espacio). Devuelve un *char **.
- ✓ **strcmp(cad1, cad2)** : compara *cad1* con *cad2*. Sería incorrecto. *cad1 != cad2*. Retorna -1 (<0), 0, 1 (>0). Variante: **strncmp(cad1, cad2, n)**.

ARREGLOS

Declaración

👉 **Sintaxis**: *tipo_dato* nombreVector[componentes];
tipo_dato nombreMatrizr[filas] [columnas];

👉 Ejemplos de declaración de variables y tipos

```
int vector[30];           // posición 0 a la 29
int matriz[20][30];       /* la primera posición es la [0][0] y la
                           ultima es la [19][29] */

typedef int Tvector[10];   // Tvector es un tipo
typedef int Tmatriz[10][10]; // Tmatriz es un tipo

Tvector v;
Tmatriz mat;
```

Cuando se pasa un vector como parámetro, el paso de parámetros es por dirección (un vector es, en realidad, un puntero en C). Por tanto, se debe tener cuidado con los efectos colaterales que se producen si, dentro de un módulo, se modifica un vector que es parámetro.

Las cadenas, arreglos y matrices no pueden pasarse por valor

REGISTROS

Declaración

```
typedef struct {
    int nro;
    char palo; } Tcarta;
Tcarta c1, c2;
```

Operación con estructuras y campos

```
c1.palo='O';
c1.nro=2;
c2=c1;
```

Punteros a estructuras

```
typedef struct {
    char palo;
    int nro; } Tcarta;

void GeneraCarta (Tcarta *);
void muestra (Tcarta);

void main() {
    Tcarta c;
    GeneraCarta(&c);
    muestra(c);
}
void GeneraCarta (Tcarta * c){
    scanf("%c %d ", &c->palo, &(*c).nro);
}
void muestra (Tcarta c){
    printf("%c %d", c.palo, c.nro);
}
```

MEMORIA DINAMICA

Ya vimos que un puntero es un tipo de datos que almacena una dirección de memoria.

Existe una dirección especial que se representa por medio de la constante NULL (definida en <stdlib.h>) y se emplea cuando se quiere indicar que un puntero no apunta a ninguna dirección.

Organización de la memoria

- Segmento de código (código del programa).
- Memoria estática (variables globales y estáticas).
- Pila (stack): Variables (locales), recursividad.
- Heap (montículo): Memoria dinámicas

Reserva y liberación de memoria

Cuando se quiere utilizar el *heap*, primero hay que reservar la memoria que se desea ocupar.

Existen dos posibilidades: función **malloc** (ANSI C) y operador **new** (C++)



Tras utilizar la memoria gestionada dinámicamente, hay que liberar el espacio reservado: función **free** (ANSI C) y operador **delete** (en C++)

Si no se libera la memoria, ese espacio de memoria no podrá volverse a utilizar.

☞ **Sintaxis:** `void *malloc ( tamaño );`

La función **malloc** (se encuentra en la librería **stdlib.h**) asigna una región de memoria para un elemento de *tamaño* bytes, y devuelve la dirección del primer byte de esa región.

Los valores almacenados en ese bloque de datos quedan indefinidos.

Se garantiza que la zona de memoria concedida no está ocupada por ninguna otra variable ni otra zona asignada por **malloc**. Si no es posible conceder el bloque (p.ej. no hay memoria suficiente), devuelve un puntero NULL.

La creación del objeto y asignación de memoria se hace en tiempo de ejecución.

Punteros void*

La función **malloc** devuelve un puntero inespecífico (no apunta a un tipo de datos determinado). En C, estos punteros sin tipo se declaran como *void**. Muchas funciones que devuelven direcciones de memoria utilizan los punteros *void**, éstos pueden convertirse a cualquier otra clase de puntero:

☞ Ejemplo

```
char *pch = (char *) malloc (100); //cadena dinámica o vector ?
int *pint = (int *) malloc(sizeof(int)); //int *pint =(int *)malloc(2);
float *pf;
pf = (float *) malloc(sizeof(float));
```

sizeof

La función **malloc** necesita conocer cuántos bytes se quieren reservar. El tamaño en bytes de un elemento de tipo T se obtiene con la expresión *sizeof (T)*.

☞ **Sintaxis:** `void free (puntero);`

Se libera el bloque correspondiente al elemento cuya dirección indica *puntero*. Si el puntero tiene valor nulo, no hace nada.

☞ Ejemplo

```
int * ptr;
ptr = (int*) malloc (sizeof(int)); //un entero
free(ptr);
ptr = (int*) malloc (30*sizeof(int)); //30 enteros
free (ptr);
```

EJERCICIO

Analizar el siguiente código considerando posibles errores y efectos no deseados

```
typedef struct {
    char palo;
    int nro; }Tcarta;
void main() {
    int * Pt1, * Pt2, X=10;
    Tcarta * PtCarta;
    char * cad;

    Pt1 = (int *) malloc(sizeof(int));
    scanf("%d",Pt1);
    Pt2 = &X; //No es manejo de memoria dinámica
    printf("\n%d %d", X, *Pt2);
    *Pt2= *Pt1 + 4;
    printf("\n%d %d",*Pt1, *Pt2);
    Pt2 = Pt1;
    printf("\n%d %d\n",*Pt1, *Pt2);
    free(Pt2);
    free(Pt1);

    PtCarta =(Tcarta *)malloc (sizeof(Tcarta));
    scanf("%d %c",&(PtCarta->nro), &((*PtCarta).palo));
    printf("\n%d %c\n",(*PtCarta).nro, PtCarta->palo);
```

```
cad = (char *)malloc(30*sizeof(char));
scanf("%s",cad);
printf("\n%c  %c",cad[0], cad[1]);
free(cad); // cad=NULL?
free(PtCarta);
}
```

ESTRUCTURAS DINÁMICAS

Las estructuras dinámicas se construyen con punteros. Un componente del elemento es un puntero que "apunta" a otro elemento del mismo tipo.

Se consigue así, un conjunto de elementos relacionados cuya cantidad puede aumentar y disminuir en tiempo de ejecución.

Las estructuras dinámicas se manejan con definiciones de datos recursivas.

Ejemplo

```
typedef struct nodo {
    char palo;
    int nro;
    struct nodo *sig;} nodo;

typedef nodo * Tlista;

Tlista L; //vble de tipo puntero a nodo
```

EJERCICIO

Crear dos nodos y enlazarlos entre sí. Colocar información en ambos nodos